Guía de Actividades Práctico-Experimentales Nro. 003

Informe de la práctica

1. Datos Generales

Nombre del estudiante(s)	Israel Rodríguez Douglas Carreño Jossibel Pérez Jorge Luzuriaga
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	003
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Martes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.

- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales, Reactivos:

- Guía práctica
- Lista de ejercicios

4. Equipos y Herramientas

Hardware

- Computador

Software

- Java 17 con Spring Boot (cada ejercicio es un microservicio)
- Maven
- Visual Studio Code
- Git y GitHub
- Navegador web para el frontend
- Sistema operativo Debian 12

5. Procedimiento / Metodología

Implementamos los cinco ejercicios de la guía [1] en Java 17 con Spring Boot. Cada ejercicio es un microservicio independiente con su propio puerto, y un frontend común en HTML, CSS y JavaScript muestra lo que hace cada hilo. Todo está en el repositorio SyncLab. Cada integrante trabajó su ejercicio en una rama de Git (feature/estacionamiento, feature/cajero, feature/productor-consumidor, feature/puente y feature/impresion) y después unimos las ramas.

Ejercicio	Microservicio	Puerto	Página
1. Estacionamiento	estacionamiento-service	8080	index.html
2. Cajero	cajero-service	8082	ejercicio2.html
3. Productor-Consumidor	fabrica-service	8083	ejercicio3.html
4. Puente	puente-service	8084	ejercicio4.html
5. Centro de impresión	impresion-service	8085	ejercicio5.html

Tabla 1: Microservicios del proyecto

Para ejecutar un servicio se entra a su carpeta y se usa mvn spring-boot:run. El frontend llama a cada servicio por su puerto, según el archivo frontend/js/config.js.

Ejercicio 1. Control de acceso a un estacionamiento (semáforo contador)

El recurso compartido son los 4 espacios del estacionamiento. En EstacionamientoService creamos un semáforo inicializado en 4 con la clase Semaphore de Java [3]. Cada vehículo es

un hilo que llama a procesarVehiculo(id): se registra como ESPERANDO, pide un espacio con acquire(), que es el wait() de la guía, pasa a DENTRO, permanece 3 segundos y al salir libera el espacio con release(), que es el signal(). El release() está en un bloque finally para que el espacio se libere aunque ocurra un error. El controlador crea los 10 hilos con new Thread.

```
private final Semaphore espacios = new Semaphore(4);

public void procesarVehiculo(int id) {
    ...
    try {
        espacios.acquire();  // wait(espacios)
        vehiculo.setEstado("DENTRO");
        ...
        Thread.sleep(3000);  // permanece estacionado
        vehiculo.setEstado("FUERA");
        ...
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        espacios.release();  // signal(espacios)
    }
}
```

Durante la simulación consultamos el estado del servicio 60 veces, cada 0,25 segundos, y el número de vehículos dentro nunca pasó de 4. En la Fig 1 se ven los 4 espacios ocupados y los otros 6 vehículos esperando en la cola.

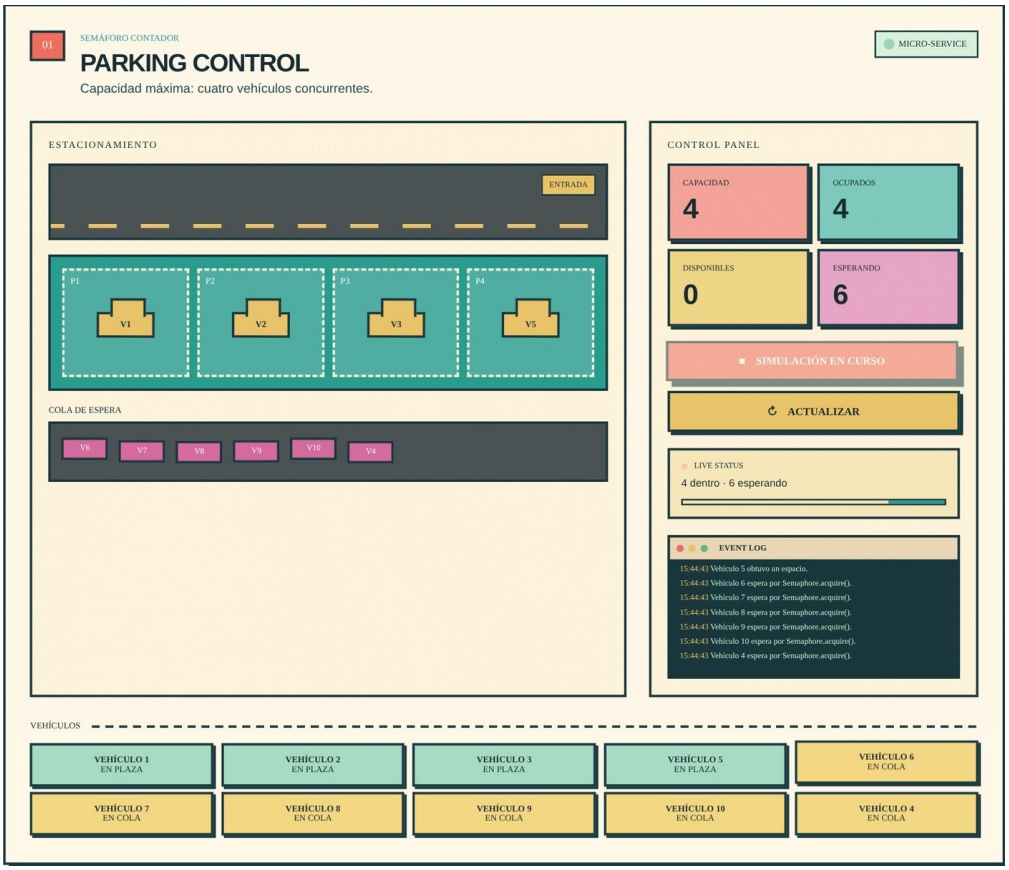


Fig 1: Verificación de que nunca ingresen más de 4 vehículos simultáneamente

Pregunta de análisis: ¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

Con el semáforo en 1 solo entraría un vehículo a la vez y el estacionamiento funcionaría como si tuviera un solo espacio. Los 10 vehículos pasarían uno por uno y, como cada uno permanece 3 segundos, la simulación tardaría unos 30 segundos en lugar de los 9 que tarda con 4 espacios (tres turnos de 4, 4 y 2 vehículos). Con el semáforo en 10 los 10 vehículos entrarían al mismo tiempo, porque hay un permiso para cada uno, y el semáforo dejaría de limitar el acceso: habría 10 vehículos dentro de un estacionamiento de 4 espacios. El valor inicial del semáforo tiene que ser igual a la capacidad real del recurso.

Ejercicio 2. Cajero automático compartido (exclusión mutua)

La variable compartida es el saldo, que empieza en $1000. En CajeroService lo protegimos con un ReentrantLock [4]. Los métodos retirar y depositar toman el mutex con lock() antes de consultar o modificar el saldo y lo sueltan con unlock() en un bloque finally. El método simular() lanza los tres procesos de la guía con un ExecutorService: retiro de $300, retiro de $250 y depósito de $200.

```
private double saldo = 1000;
private final ReentrantLock mutexSaldo = new ReentrantLock();

public void retirar(int id, double cantidad) {
    ...
    mutexSaldo.lock();  // lock(mutex_saldo)
    try {
        ...
        if (saldo >= cantidad) {
            saldo = saldo - cantidad;
            ...
        } else {
            ...
        }
    } finally {
        mutexSaldo.unlock();  // unlock(mutex_saldo)
    }
}
```

Para comparar hicimos también CajeroSinMutexService, que realiza las mismas operaciones sin el mutex. Cada hilo copia el saldo en una variable temporal, espera 100 ms y después escribe el nuevo valor. Esa pausa hace que los tres hilos lean el saldo antes de que los otros lo actualicen.

Ejecutamos cada versión 5 veces. Con mutex el saldo final fue $650 en las 5 ejecuciones. Sin mutex los resultados fueron $700, $750, $750, $1200 y $1200, ninguno correcto. La Fig 2 muestra una de las comparaciones.



Fig 2: Verificación de que el saldo no varíe en cada ejecución (con mutex $650, sin mutex $750)

Pregunta de análisis: ¿Por qué el acceso a saldo debe protegerse con un mutex?

Porque retirar o depositar no es una sola instrucción: el hilo lee el saldo, calcula el nuevo valor y lo escribe. Si dos hilos leen el mismo saldo antes de que alguno escriba, el último que escribe borra la operación del otro. Eso pasó en las ejecuciones sin mutex. Cuando salió $1200, los tres hilos leyeron $1000 y el depósito fue el último en escribir (1000 + 200), así que los dos retiros se perdieron. Con el mutex solo un hilo a la vez puede estar entre la lectura y la escritura, y el resultado siempre es 1000 − 300 − 250 + 200 = $650.

Ejercicio 3. Fábrica de ensamblaje (Productor-Consumidor)

La clase Almacen tiene la cola con capacidad para 5 piezas, el semáforo espacios inicializado en 5, el semáforo elementos inicializado en 0 y un ReentrantLock que protege la cola. En Java el wait() de la guía es tryAcquire() y el signal() es release(). EstrategiaConcurrente crea un hilo por cada productor y por cada consumidor (2 y 2) y cada uno procesa 5 piezas.

En lugar de acquire() usamos tryAcquire() con un tiempo límite de 2 segundos. Con los valores de la guía nunca se llega a ese límite, pero con espacios = 0 el hilo se quedaría esperando para siempre y el servidor no respondería. Con el límite podemos detectar el bloqueo y mostrarlo en la página.

```
public Resultado insertar(Pieza pieza, String actor) throws InterruptedException {
    if (!espacios.tryAcquire()) {  // wait(espacios)
        ...
```

```
        if (!espacios.tryAcquire(timeoutMs, TimeUnit.MILLISECONDS)) {
            ...
            return Resultado.BLOQUEADO;
        }
    }
    Resultado r = Resultado.OK;
    mutex.lock();  // lock(mutex_almacen)
    try {
        cola.addLast(pieza);  // insertar(almacen, pieza)
        ...
    } finally {
        mutex.unlock();  // unlock(mutex_almacen)
    }
    elementos.release();  // signal(elementos)
    return r;
}
```

El consumidor, en el método extraer(), hace lo contrario: tryAcquire() sobre elementos, toma el mutex, saca la pieza con pollFirst(), suelta el mutex y hace release() sobre espacios. Cada hilo publica lo que hace y la página lo recibe en vivo por Server-Sent Events. En la Fig 3 se ve el almacén con piezas durante la ejecución.

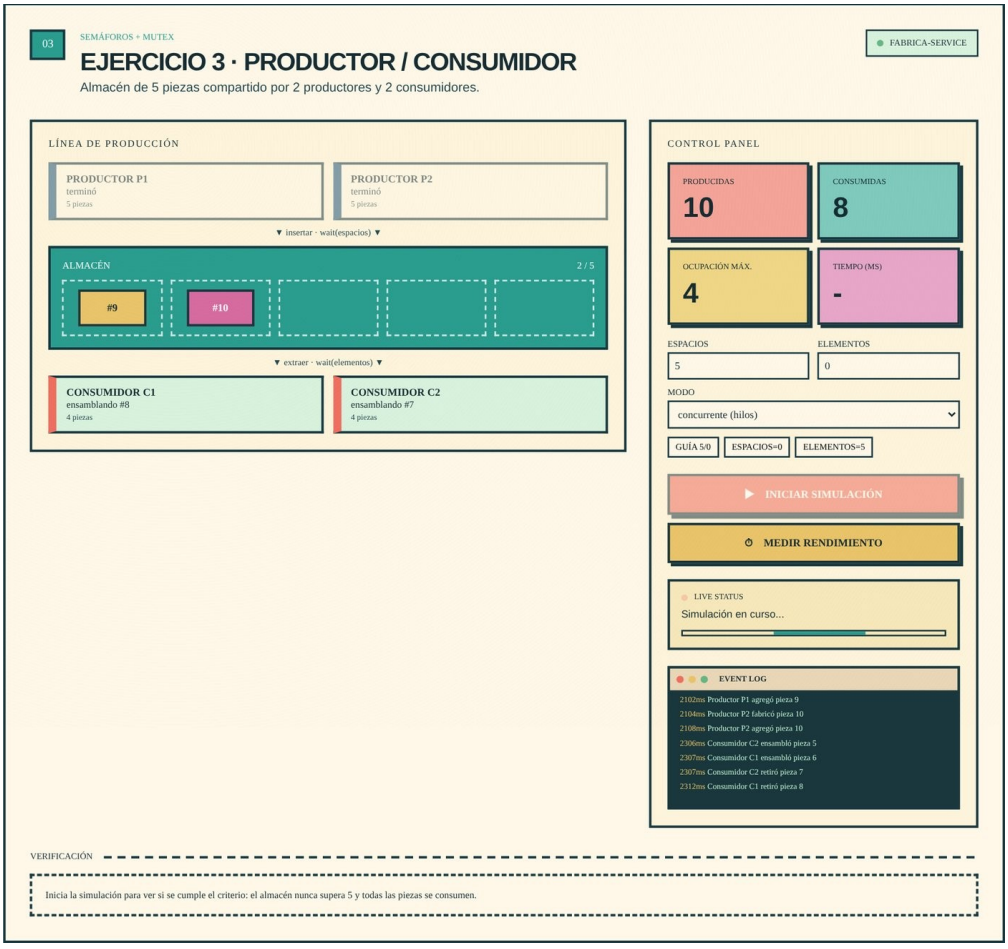


Fig 3: Productores y consumidores trabajando sobre el almacén

Al terminar, la página revisa el criterio de la guía: se consumieron las 10 piezas producidas, la ocupación máxima del almacén fue 4 y no hubo inconsistencias (Fig 4).

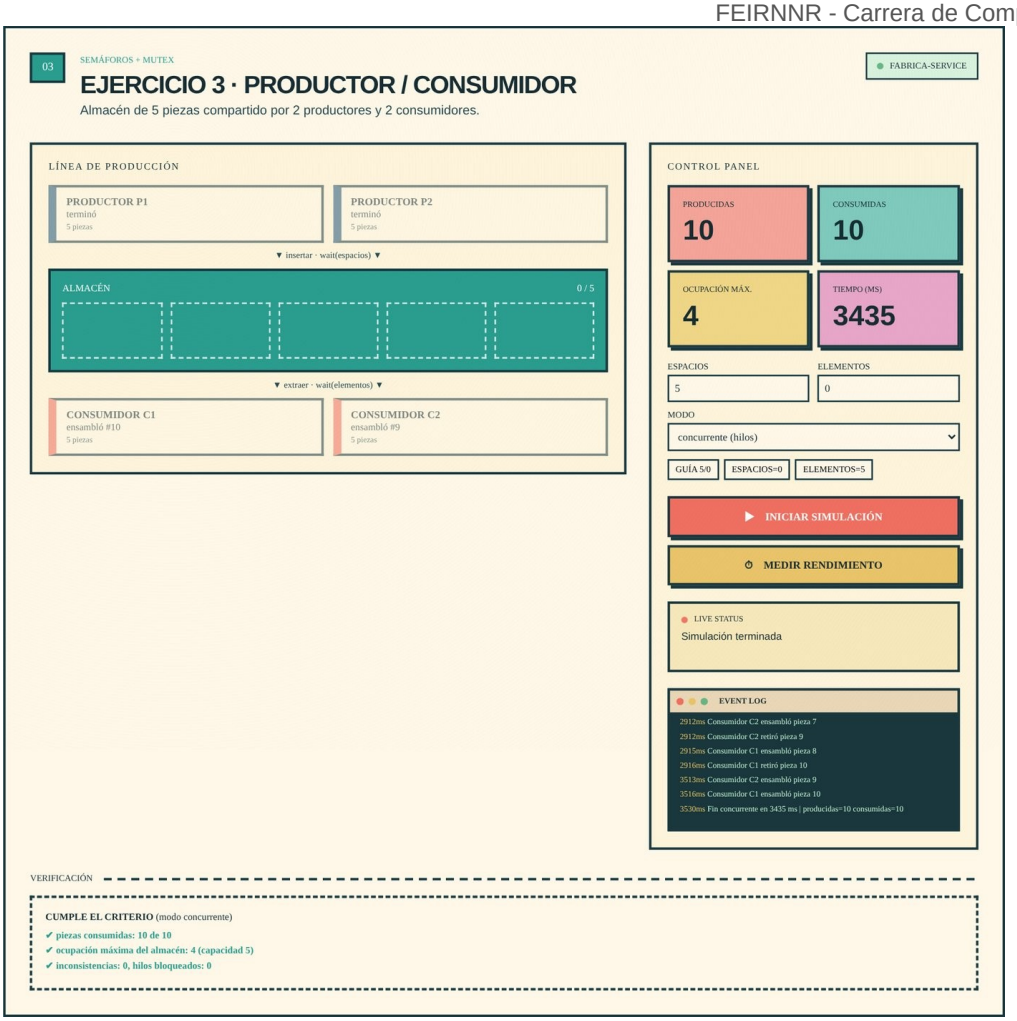


Fig 4: Verificación de que el almacén nunca supere 5 productos ni quede con inconsistencias

Pregunta de análisis: ¿Qué sucede si espacios se inicializa en 0? ¿Qué sucede si elementos se inicializa en 5?

Con espacios = 0 el sistema cree que el almacén está lleno aunque está vacío. Los productores se quedan en wait(espacios) y nunca insertan una pieza, y los consumidores se quedan en wait(elementos) porque nunca llega ninguna. Es un interbloqueo: en la prueba se produjeron 0 piezas y los 4 hilos quedaron bloqueados (Fig 5).

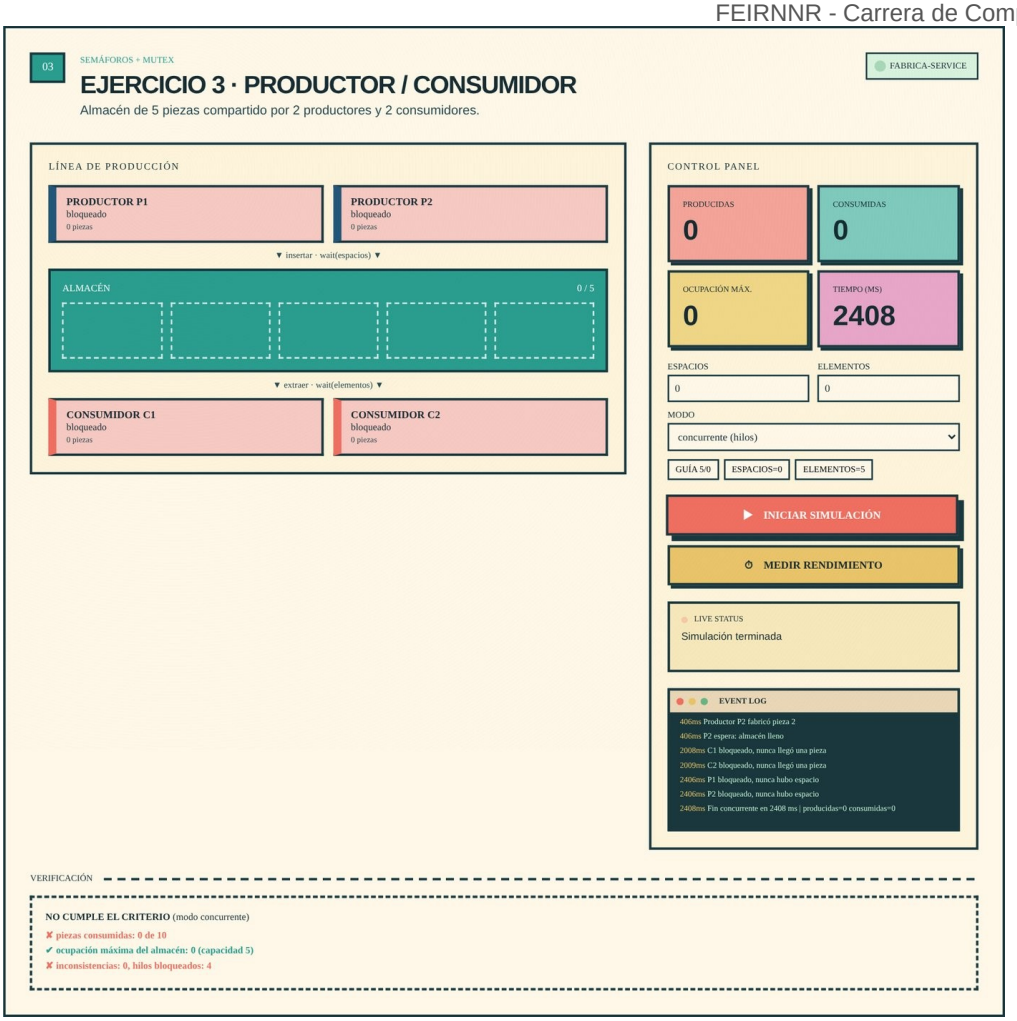


Fig 5: Interbloqueo con espacios = 0

Con elementos = 5 el sistema cree que hay 5 piezas cuando el almacén está vacío. Los consumidores pasan wait(elementos) sin esperar e intentan retirar piezas que no existen. Además, cada retiro falso hace signal(espacios) de más, y los productores llegan a meter más piezas de las que caben. En la prueba solo se consumieron 5 de las 10 piezas, hubo 6 inconsistencias y el almacén llegó a tener 6 piezas con capacidad 5 (Fig 6). Estos números cambian entre ejecuciones porque dependen del orden en que el sistema operativo ejecuta los hilos.

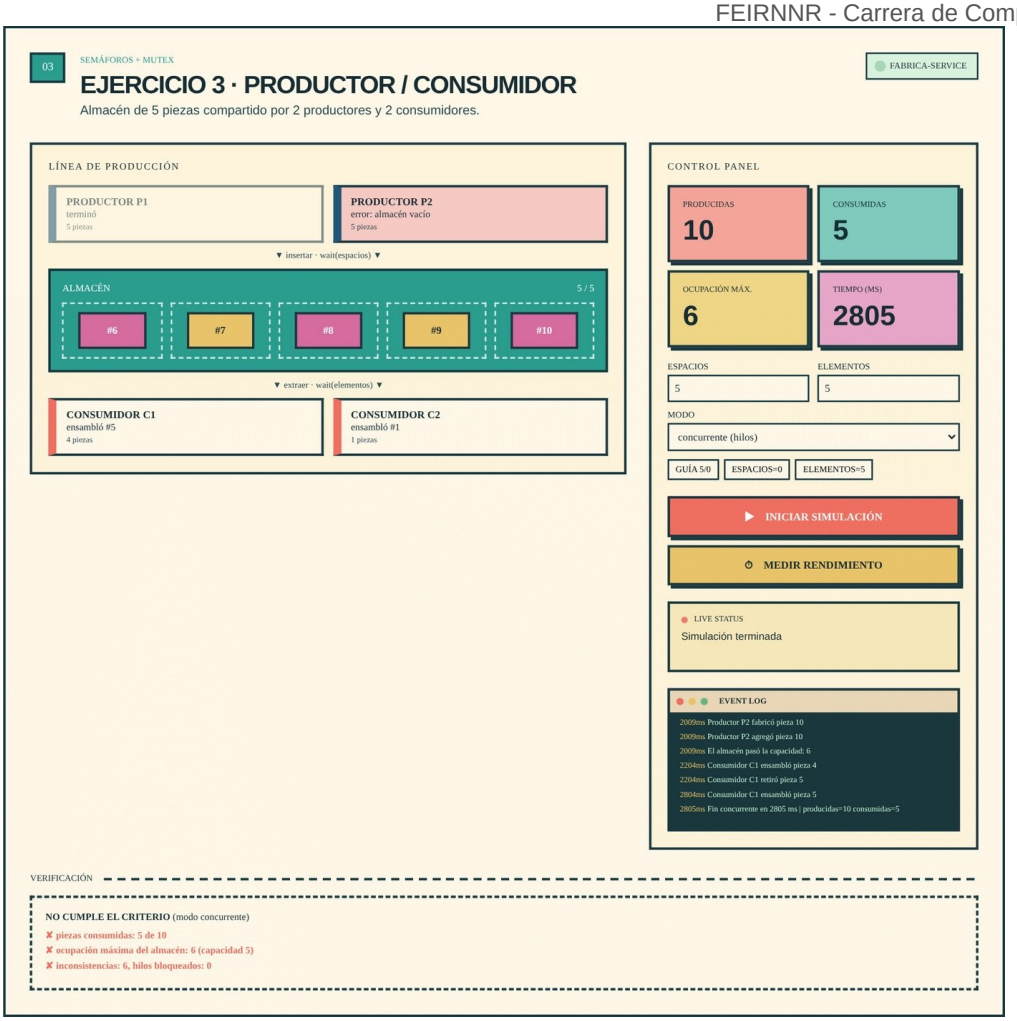


Fig 6: Inconsistencias con elementos = 5

Para medir el rendimiento hicimos también EstrategiaSecuencial, donde un solo hilo hace el mismo trabajo pieza por pieza. Con 5 repeticiones de cada versión, la secuencial tardó en promedio 2504 ms y la concurrente 852,6 ms (Fig 7).

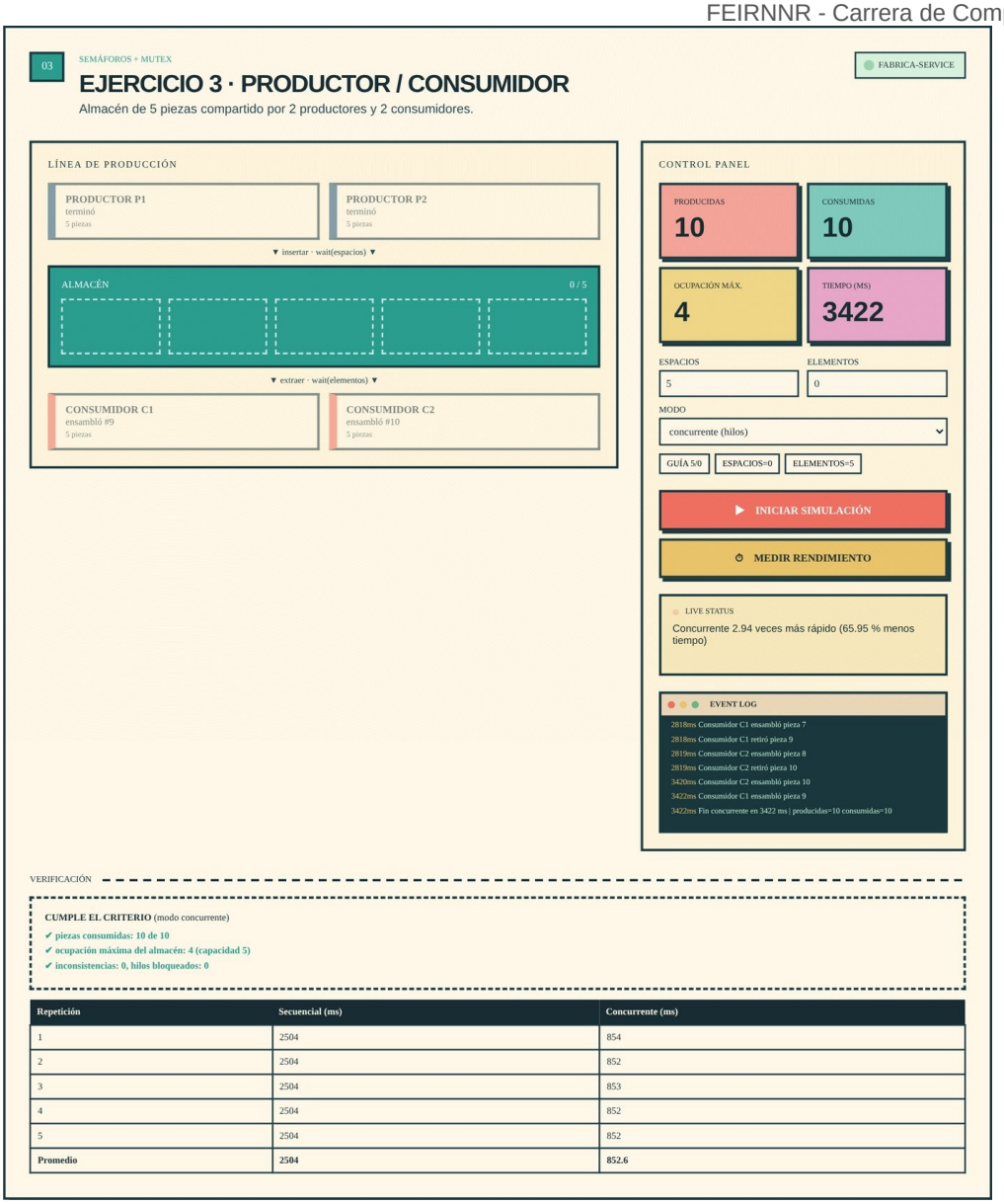


Fig 7: Comparación de rendimiento secuencial vs concurrente del Ejercicio 3

Ejercicio 4. Cruce de vehículos (control de acceso en una dirección)

El recurso compartido es el puente. La clase Puente guarda la dirección actual (NORTE, SUR o NINGUNA) y los contadores vehiculosNorte y vehiculosSur, y los protege con un ReentrantLock. Cada vehículo es un hilo que llama a entrar(), duerme el tiempo de cruce y llama a salir().

En el pseudocódigo de la guía el esperar() está dentro del lock(mutex). Si el vehículo esperara sin soltar el mutex, ningún vehículo podría entrar a salir() para poner la dirección en NINGUNA y el puente se congelaría. Por eso usamos una variable de condición (Condition [5]): await() suelta el mutex mientras el hilo espera y lo vuelve a tomar al despertar, y signalAll() despierta a todos los que esperan cuando sale un vehículo.

```
public long entrar(Vehiculo v) throws InterruptedException {
    ...
    mutex.lock();
    try {
```

```
        cambiarEsperando(d, +1);
        ...
        while (!politica.puedeEntrar(d, this)) {  // MIENTRAS direccion == contraria
            ...
            condicion.await();  // esperar(), suelta el mutex
        }
        cambiarEsperando(d, -1);
        if (direccion != d) {
            direccion = d;
            consecutivos = 0;
            ...
        }
        consecutivos++;
        cambiarEnPuente(d, +1);  // vehiculos_d = vehiculos_d + 1
        ...
    } finally {
        mutex.unlock();
    }
    ...
}

public void salir(Vehiculo v) {
    mutex.lock();
    try {
        cambiarEnPuente(d, -1);  // vehiculos_d = vehiculos_d - 1
        ...
        if (enPuente(d) == 0) {
            politica.alQuedarLibre(d, this);  // asigna el turno (reto)
            direccion = Direccion.NINGUNA;
            ...
        }
        condicion.signalAll();
    } finally {
        mutex.unlock();
    }
}
```

Los métodos entrar() y salir() reciben la dirección del vehículo, así que el mismo código sirve para el norte y para el sur. El procedimiento vehiculo_sur(id) que pide la guía queda así:

```
PROCEDIMIENTO vehiculo_sur(id)
    lock(mutex)
    MIENTRAS direccion == NORTE HACER
        esperar(condicion)
    FIN MIENTRAS
    direccion = SUR
    vehiculos_sur = vehiculos_sur + 1
    unlock(mutex)

    escribir("Vehículo ", id, " cruza hacia el SUR")
    cruzar_puente(id)

    lock(mutex)
    vehiculos_sur = vehiculos_sur - 1
    SI vehiculos_sur == 0 ENTONCES
        direccion = NINGUNA
    FIN SI
```

```
    despertar_a_todos(condicion)
    unlock(mutex)
FIN PROCEDIMIENTO
```

Cada vez que entra un vehículo revisamos si hay vehículos en el sentido contrario, y si los hay contamos una violación. En todas las ejecuciones el contador quedó en 0 (Fig 8).

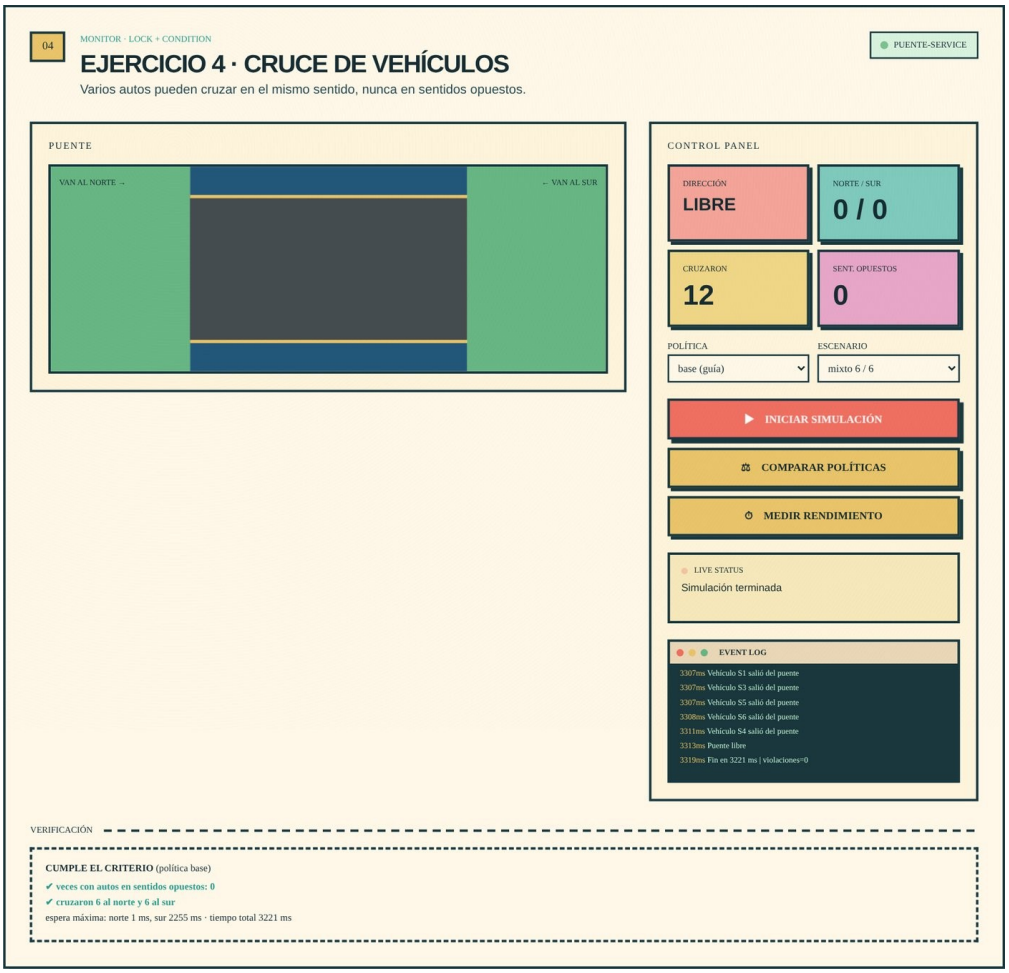


Fig 8: Verificación de que nunca crucen vehículos en sentidos opuestos simultáneamente

Análisis de la inanición. Con la regla de la guía un vehículo entra si el puente está libre o si ya va en su dirección. Para probarlo armamos un escenario con 15 vehículos hacia el norte y 3 hacia el sur. Mientras siguen llegando vehículos al norte, el puente nunca queda vacío y los del sur esperan (Fig 9). En la prueba el sur esperó hasta 1421 ms y el norte 0 ms.

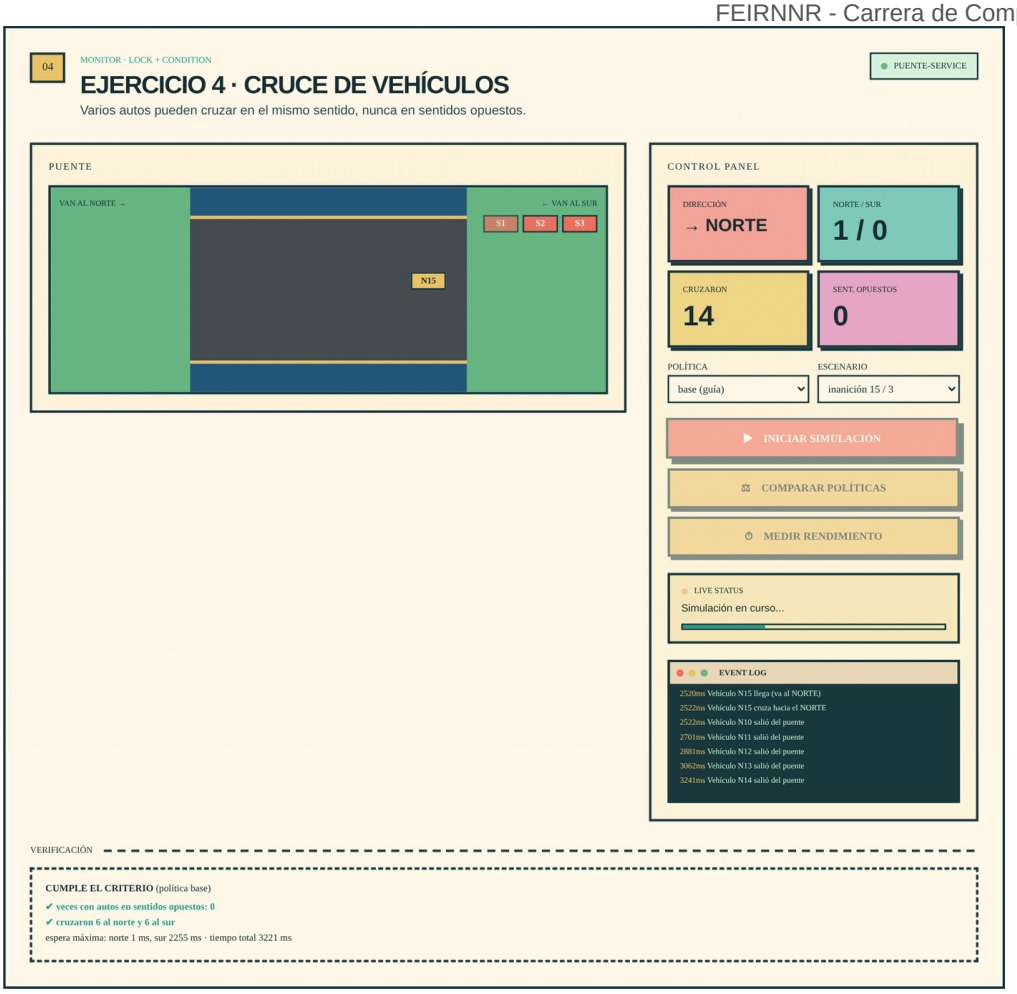


Fig 9: Inanición del sur con la política de la guía

Reto. Para que una dirección no monopolice el puente hicimos PoliticaJusta. Si alguien espera del otro lado, solo pueden entrar 3 vehículos seguidos en la misma dirección. Cuando el puente se vacía, el turno pasa al lado que estaba esperando. Si nadie espera enfrente, los vehículos siguen pasando sin límite.

```
public boolean puedeEntrar(Direccion d, Puente p) {
    boolean alguienEnfrente = p.esperando(d.contraria()) > 0;
    if (p.getDireccion() == Direccion.NINGUNA) {
        return p.getTurno() == Direccion.NINGUNA || p.getTurno() == d || !
alguienEnfrente;
    }
    if (p.getDireccion() == d) {
        return !alguienEnfrente || p.getConsecutivos() < limite;  // limite = 3
    }
    return false;
}
```

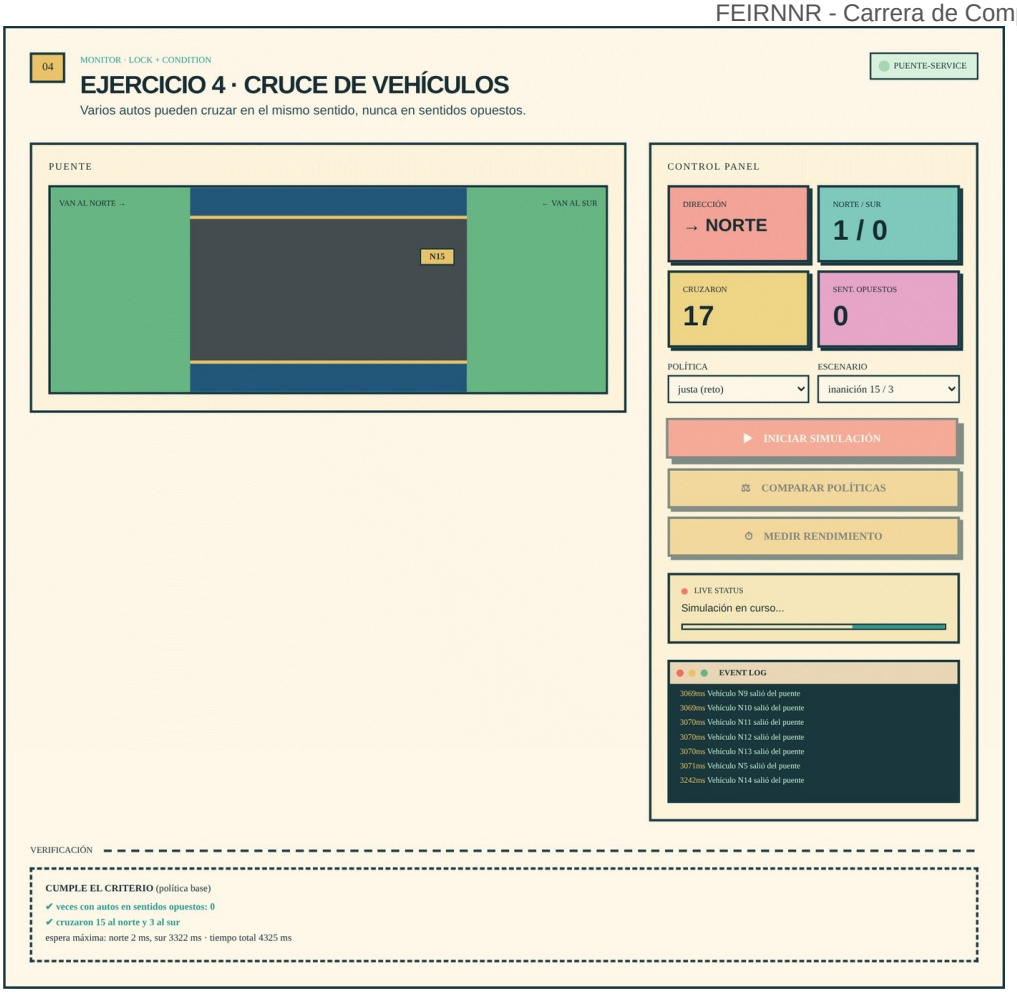
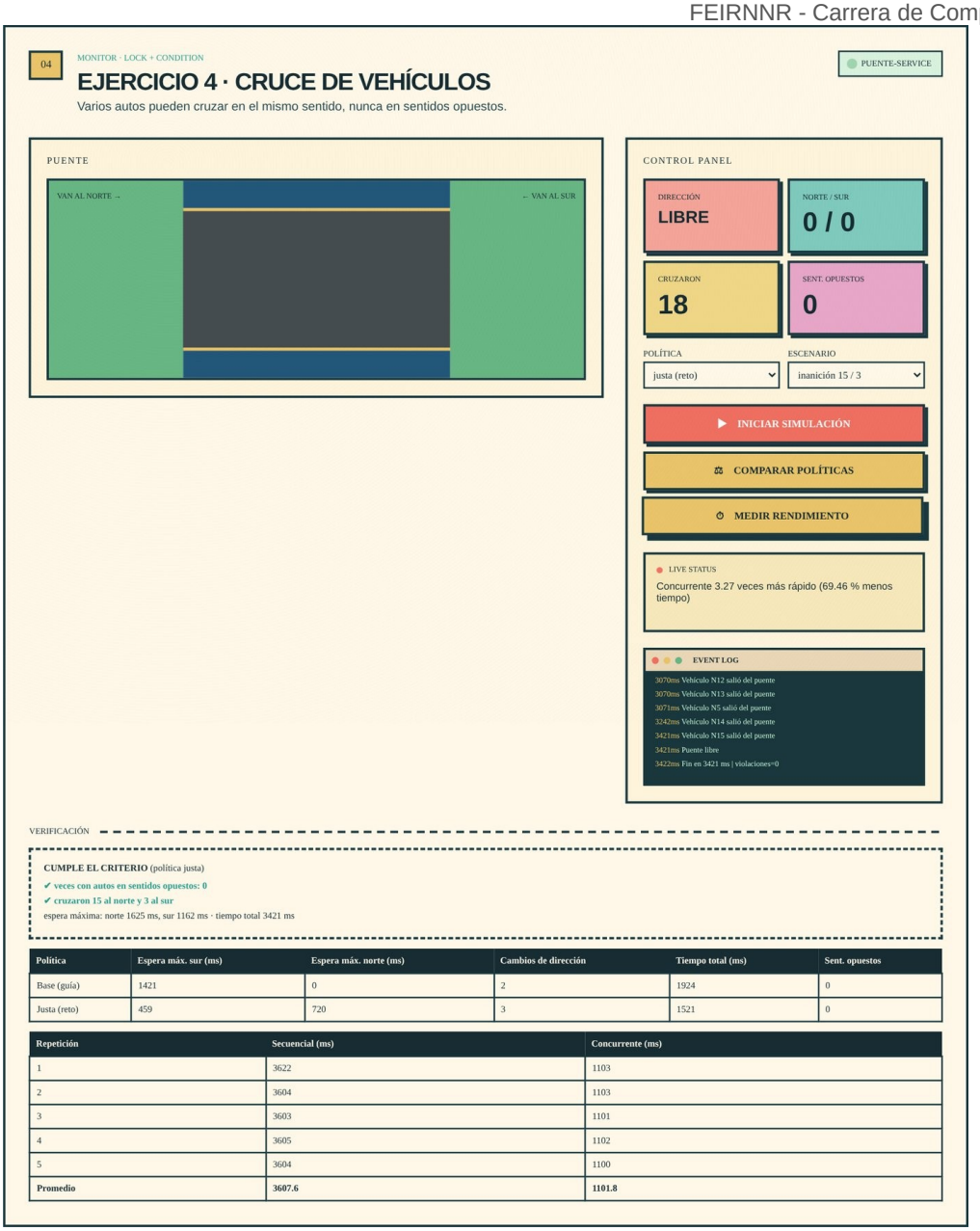


Fig 10: Política justa: los vehículos del sur entran antes

Con la política justa la espera máxima del sur bajó de 1421 ms a 459 ms, un 67,7 % menos. A cambio, el norte pasó a esperar hasta 720 ms y el puente cambió de dirección 3 veces en lugar de 2. En las dos políticas no hubo vehículos en sentidos opuestos. La Fig 11 muestra la comparación y la medición de rendimiento: la versión secuencial, donde cruza un vehículo a la vez, tardó en promedio 3607,6 ms y la concurrente 1101,8 ms.



Política	Espera máx. sur (ms)	Espera máx. norte (ms)	Cambios de dirección	Tiempo total (ms)	Sent. opuestos
Base (guía)	1421	0	2	1924	0
Justa (reto)	459	720	3	1521	0

Repetición	Secuencial (ms)	Concurrente (ms)
1	3622	1103
2	3604	1103
3	3603	1101
4	3605	1102
5	3604	1100
Promedio	**3607.6**	**1101.8**

Fig 11: Comparación de políticas y rendimiento del Ejercicio 4

Ejercicio 5. Centro de impresión (sincronización de procesos)

En SincronizadorService hay un semáforo para cada etapa: turnoDiseno empieza en 1, para que Diseño pueda arrancar, y turnoRevision y turnoImpresion empiezan en 0. Cada etapa es una clase que implementa el patrón Command (DisenoEtapa, RevisionEtapa e ImpresionEtapa) y corre en su propio hilo. Cada una repite 3 veces lo mismo: espera su turno con acquire(), hace su trabajo durante 1,5 segundos y le da el turno a la siguiente etapa con release(). Impresión le devuelve el turno a Diseño.

```
private Semaphore turnoDiseno = new Semaphore(1);
private Semaphore turnoRevision = new Semaphore(0);
private Semaphore turnoImpresion = new Semaphore(0);

// RevisionEtapa
public void ejecutarEtapa() throws InterruptedException {
```

```
    sincronizador.getTurnoRevision().acquire();  // wait(turno_revision)
    ...
    CentroImpresionController.agregarLog("🔍 [REVISIÓN] Verificando documento...");
    Thread.sleep(1500);
    sincronizador.getTurnoImpresion().release();  // signal(turno_impresion)
}
```

Ejecutamos la simulación 3 veces y revisamos el registro del servicio. En las 3 ejecuciones aparecieron 9 etapas en el orden Diseño, Revisión, Impresión repetido 3 veces, sin que ninguna etapa se adelantara (Fig 12).

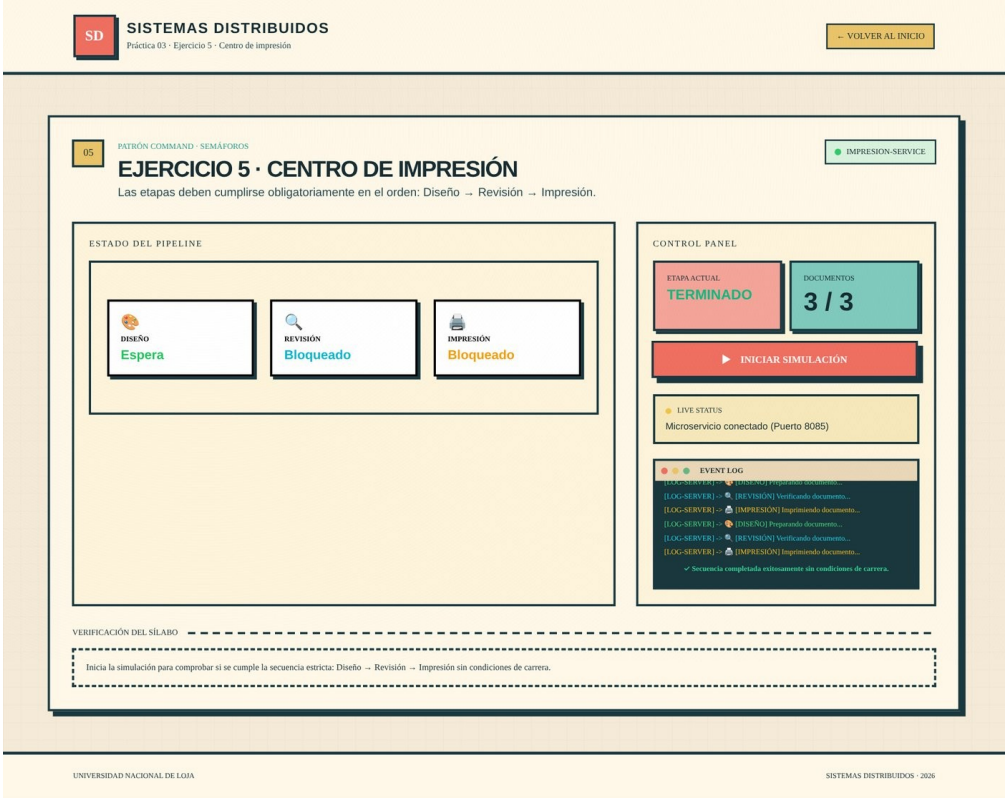


Fig 12: Verificación de que ningún proceso avance a la siguiente etapa antes de tiempo

Análisis: si se elimina uno de los signal(), la etapa que debía recibir el turno se queda esperando para siempre. Por ejemplo, si Diseño no hace signal(turno_revision), Revisión nunca pasa su acquire(), Impresión tampoco recibe su turno y Diseño, después de preparar el primer documento, se queda bloqueado esperando un turno que nadie le devuelve. El resultado es un interbloqueo con un solo documento diseñado y ninguno impreso.

6. Resultados esperados:

- Comprobación de las distintas formas de cómo los hilos se comunican.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes.
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica.

En la práctica los hilos se comunican de tres formas. En los ejercicios 1, 3 y 5 se avisan por medio de semáforos: un hilo hace release() y otro que estaba en acquire() continúa. En el

ejercicio 2 comparten una variable protegida con un mutex. En el ejercicio 4 usan un monitor, donde los hilos esperan con await() y se despiertan con signalAll(). Además, cada microservicio se comunica con el navegador por HTTP, y en los ejercicios 3 y 4 los eventos de los hilos llegan en vivo por Server-Sent Events.

Ejercicio	Resultado esperado	Criterio de verificación	Resultado obtenido	Estado
1. Estacionamiento	Vehículos dentro ≤ 4	Verificar que nunca ingresen más de 4 simultáneamente	En 60 lecturas durante la simulación el máximo de vehículos dentro fue 4.	Cumple
2. Cajero	Saldo final = $650	Ejecutar varias veces y comprobar que el saldo no varíe	Con mutex, $650 en las 5 ejecuciones. Sin mutex el saldo varió entre $700 y $1200.	Cumple
3. Productor-Consumidor	Todos los productos producidos son consumidos	Verificar que el almacén nunca supere 5 ni quede con inconsistencias	10 piezas producidas y 10 consumidas, ocupación máxima 4 y 0 inconsistencias.	Cumple
4. Puente	Solo una dirección activa a la vez	Comprobar que nunca crucen vehículos en sentidos opuestos simultáneamente	0 casos de vehículos en sentidos opuestos con las dos políticas.	Cumple
5. Centro de Impresión	Diseño → Revisión → Impresión	Verificar que ningún proceso avance a la siguiente etapa antes de tiempo	En 3 ejecuciones el orden fue Diseño, Revisión, Impresión para los 3 documentos.	Cumple

Tabla 2: Verificación de los resultados esperados

Medimos el rendimiento en los ejercicios 3 y 4, que tienen una versión secuencial y otra concurrente que hacen el mismo trabajo. Cada versión se ejecutó 5 veces.

Ejercicio	Secuencial (ms)	Concurrente (ms)	Aceleración	Reducción
3. Productor-Consumidor	2504	852,6	2,94 veces	65,95 %
4. Puente	3607,6	1101,8	3,27 veces	69,46 %

Tabla 3: Rendimiento secuencial vs concurrente (promedio de 5 ejecuciones)

La versión concurrente es más rápida porque las tareas se solapan: mientras un productor fabrica, un consumidor ensambla otra pieza, y en el puente cruzan varios vehículos del mismo sentido a la vez. La mejora no llega a 4 veces aunque haya 4 hilos, porque la sección crítica protegida por el mutex y las esperas en los semáforos no se pueden hacer en paralelo.

Enlace al repositorio en GitHub: https://github.com/Doomblas/SyncLab.git

Enlace al video de explicación: [agrega aquí el enlace del video]

7. Conclusiones:

- El semáforo contador del ejercicio 1 limitó el estacionamiento a 4 vehículos: en todas las lecturas hubo como máximo 4 dentro mientras los demás esperaban. El valor inicial tiene que ser igual a la capacidad real, porque con 1 se pierde la concurrencia y con 10 el semáforo deja de proteger el recurso.
- Sin el mutex el saldo del cajero salió distinto en cada ejecución ($700, $750 y hasta $1200), porque los hilos leen el mismo saldo y el último en escribir borra la operación de los otros. Con el mutex el resultado fue $650 en todas las ejecuciones.
- En el productor-consumidor los dos semáforos controlan cuándo puede avanzar cada hilo y el mutex protege la cola. Con los valores de la guía se consumieron las 10 piezas, pero con espacios = 0 hubo un interbloqueo de los 4 hilos y con elementos = 5 se leyeron piezas que no existían, así que los valores iniciales tienen que reflejar el estado real del almacén.
- En el puente la espera tiene que soltar el mutex, por eso usamos await() de una variable de condición. La regla de la guía es segura, pero con un flujo continuo hacia el norte el sur esperó 1421 ms; con el límite de 3 vehículos seguidos la espera del sur bajó a 459 ms, a cambio de que el norte espere más y el puente cambie más veces de dirección.
- Con tres semáforos encadenados el centro de impresión respetó siempre el orden Diseño, Revisión, Impresión. Basta con quitar un solo signal() para que la cadena se detenga.
- Las versiones concurrentes fueron 2,94 veces más rápidas en el ejercicio 3 y 3,27 veces en el ejercicio 4 que las secuenciales. Separar cada ejercicio en un microservicio permitió que cada integrante trabajara su parte en su propia rama y que cada servicio se pueda ejecutar y probar por separado.

8. Preguntas de Control

- **¿Qué es una condición de carrera?**

Es cuando dos o más hilos usan un dato compartido al mismo tiempo y el resultado depende del orden en que se ejecutan [2]. En el ejercicio 2 sin mutex los tres procesos leyeron el saldo antes de que los otros lo actualizaran, y por eso el saldo final cambió entre ejecuciones ($700, $750, $1200) en lugar de ser siempre $650.

- **¿Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente?**

No se pueden usar siempre de la misma forma. Estas son las diferencias prácticas que vimos en la práctica:

1. Dueño: el mutex solo lo puede liberar el hilo que lo tomó; un semáforo lo puede liberar cualquier hilo. En el ejercicio 3 el productor hace release() de elementos y es el consumidor el que lo toma, algo que no se podría hacer con un mutex.
2. Uso: el mutex sirve para exclusión mutua, para que un solo hilo entre a la sección crítica, como el saldo del ejercicio 2. El semáforo sirve para señalizar entre hilos y contar recursos, como los espacios del ejercicio 1 o los turnos del ejercicio 5.

3. Reentrada: un ReentrantLock deja que el mismo hilo lo tome varias veces [4]; si un hilo hace dos veces wait() sobre un semáforo binario en 1, se bloquea a sí mismo.

Un semáforo binario puede imitar a un mutex para proteger una sección crítica, pero no al revés: el mutex no sirve para avisar entre hilos distintos.

- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**

Un interbloqueo ocurre cuando dos o más hilos quedan esperando para siempre porque cada uno espera algo que tiene otro. Según Coffman [6] deben cumplirse las cuatro condiciones a la vez:

1. Exclusión mutua: el recurso solo lo puede usar un hilo a la vez.
2. Retención y espera: un hilo tiene un recurso y espera otro.
3. No apropiación: no se le puede quitar el recurso a un hilo a la fuerza.
4. Espera circular: existe una cadena de hilos donde cada uno espera un recurso que tiene el siguiente.

El ordenamiento de recursos elimina la espera circular: si todos los hilos piden los recursos en el mismo orden, no se puede formar un ciclo. En la práctica vimos interbloqueos en el ejercicio 3 con espacios = 0, donde productores y consumidores se esperaban entre sí, y en el análisis del ejercicio 5 al quitar un signal().

9. Declaración de uso de IA

Para el desarrollo de la práctica se utilizó Claude (Anthropic) como asistente para implementar y depurar los ejercicios 3 y 4 (fabrica-service y puente-service), integrarlos al frontend común del repositorio SyncLab y revisar la redacción del informe. Las capturas del informe se tomaron ejecutando el código del repositorio y el grupo revisó el funcionamiento de cada ejercicio.

10. Bibliografía

[1] J. Guamán, "Guía de Actividades Práctico-Experimentales Nro. 003: Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia," Sistemas Distribuidos, Semana 3, Universidad Nacional de Loja, Loja, Ecuador, 2026.

[2] A. S. Tanenbaum y M. van Steen, Distributed Systems, 4th ed. Maarten van Steen, 2023.

[3] Oracle, "Class Semaphore," Java SE 17 API Documentation. [En línea]. Disponible: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/Semaphore.html

[4] Oracle, "Class ReentrantLock," Java SE 17 API Documentation. [En línea]. Disponible: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/locks/ReentrantLock.html

[5] Oracle, "Interface Condition," Java SE 17 API Documentation. [En línea]. Disponible: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/locks/Condition.html

[6] E. G. Coffman, M. Elphick y A. Shoshani, "System Deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, 1971.